

Highlights

DriftJax: A Structure-Aware Differentiable Drift–Diffusion Solver for Photovoltaic Device Design

Mohammed Benaissa, Abd Elouahab Noua, Mohamed Maouadj

- Analytical block Jacobians support linear-cost Newton steps.
- A dense implicit adjoint makes backward-pass complexity explicit.
- Verification separates residual accuracy from sensitivity accuracy.
- Design examples link sensitivities, optimisation, and synthetic recovery.

DriftJax: A Structure-Aware Differentiable Drift–Diffusion Solver for Photovoltaic Device Design

Mohammed Benaissa^a, Abd Elouahab Noua^{a,b,*} and Mohamed Maouadj^a

^a*Research Center in Semiconductor Technology for Energetics (CRTSE), 02 Bd. Frantz Fanon, B.P.*

140, Algiers, 16038, Algeria

^b*Department of Sciences and Technology, University Larbi Ben M'hidi, Oum El Bouaghi, 04000, Algeria*

ARTICLE INFO

Keywords:

photovoltaic devices
drift–diffusion
implicit differentiation
Scharfetter–Gummel scheme
adjoint sensitivity
scientific software

ABSTRACT

We present DriftJax, a one-dimensional drift–diffusion–Poisson framework for photovoltaic simulation and design sensitivity analysis. The numerical method combines Scharfetter–Gummel fluxes with an analytically assembled block-tridiagonal Jacobian and a direct Block-Thomas Newton step. For a mesh of N nodes, the forward linear solve requires $O(N)$ work and storage. Sensitivities of converged steady-state solutions are evaluated using the implicit function theorem and a custom reverse-mode rule in JAX. The production adjoint uses pivoted dense LU, with $O(N^3)$ factorisation work and $O(N^2)$ matrix storage per bias; checkpointing reduces retained intermediates without changing this scaling. This separation of forward and backward linear algebra makes the computational trade-off explicit. The framework combines carrier-statistics, optical-generation, and recombination models with a common device interface. Reported verification studies compare analytical Jacobians, linear residuals, conservation identities, and finite-difference sensitivities. Illustrative applications address derivative-based sensitivity analysis, constrained band-gap/thickness optimisation, and recovery of a synthetic current–voltage curve. The results support the use of structured forward computation within a differentiable design workflow, while also identifying the importance of conditioning, objective normalisation, and convergence checks. The reported timing ratios are specific to the matched configurations and have not been isolated by controlled ablation; the available release records do not constitute a fully reproducible benchmark archive.

Program summary

Program title	DriftJax
Version described	v0.1.15, release notes dated 14 September 2026
Programming language	Python with JAX
Licensing provisions	MIT, as specified by the release documentation
Nature of problem	Coupled electrostatics and carrier transport in layered photovoltaic devices, including sensitivities of scalar design objectives
Solution method	Scharfetter–Gummel discretisation, analytical block Jacobian, Block-Thomas Newton solve, and dense-LU implicit adjoint
Precision	Float64 reference calculations; optional mixed-precision dense factorisation with iterative refinement
Restrictions	One spatial dimension, prescribed temperature, fixed design topology, and dense backward-pass scaling
Availability	Repository information and evidence scope are stated in the data and code availability section

*Corresponding author: Abd Elouahab Noua.

1. Introduction

Photovoltaic device design involves coupled choices of absorber properties, layer geometry, doping, and contact selectivity. Drift–diffusion simulation links these choices to terminal current–voltage characteristics through a continuum description of electrostatics, transport, generation, and recombination [1, 2]. Established tools include PC1D [3], SCAPS [4], wxAMPS [5], and Sesame [1]. Their physical scope and dimensionality differ, but each illustrates the value of solving the transport problem rather than relying solely on an equivalent-circuit description.

The corresponding design problem requires more than a forward solution. For a vector of design parameters θ , a simulation determines a state $\mathbf{u}(\theta)$ and an objective such as power-conversion efficiency $L(\mathbf{u}(\theta), \theta)$. Central finite differences of P parameters typically require $2P$ perturbed simulations for one gradient. An adjoint formulation instead evaluates the derivative of a scalar objective through a linearised system at the converged state [2, 6]. This changes the number of state solves needed for sensitivity analysis, although parameter-dependent assembly and contraction costs remain.

Mann et al. introduced an end-to-end differentiable photovoltaic simulator, ∂ PV, that combines JAX automatic differentiation with the implicit function theorem [2]. Their work establishes the model-to-gradient-to-design workflow on which the present study builds. In the v3 preprint of that article, the linear solver is described as GMRES with an ILU(0) preconditioner and compact band storage (Appendix E of Ref. [2]). It is therefore important to distinguish that published algorithm from any separately timed code revision or compatibility-modified benchmark implementation.

DriftJax addresses a specific numerical opportunity in the one-dimensional problem: each nodal residual couples only to neighbouring nodes. With three unknowns per node, the interleaved Jacobian consists of 3×3 blocks on three block diagonals. Analytical assembly and a direct Block-Thomas solve exploit this structure during Newton iteration. The production backward pass makes a different choice: it uses pivoted dense LU after earlier structured-transpose experiments exhibited gradient discrepancies on ill-conditioned systems.

The article is organised around three questions:

1. **Correctness:** Does the implementation solve the stated equations and compute the intended derivatives? (Sections 2–4.)
2. **Performance:** What computational benefit remains after accuracy and execution conditions are matched? (Section 6.)
3. **Usefulness:** Do those derivatives improve a specified design task? (Section 5.)

The contribution is not a new adjoint principle or a uniformly linear-cost differentiable solver. It is an implementation that combines structured forward algebra, an explicit backward stability strategy, and derivative verification. Detailed derivations, auxiliary solvers, and reproduction requirements are provided in Supplementary Sections S1–S9. Detailed derivations, auxiliary solvers, and reproduction requirements are provided in Supplementary Sections S1–S9. Implementation statements refer to the supplied DriftJax v0.1.15 documentation unless a reference-code inspection is explicitly identified; numerical examples are retained results whose provenance and limitations are specified with the comparisons.

1.1. Relationship to ∂ PV

Table 1 distinguishes shared methodology from implementation choices. The reference-code observations concern the public `master` branch of `romanodev/deltapv`, inspected on 14 September 2026 [7]; they do not identify the revision used for the archived timing measurements. DriftJax entries describe the supplied release documentation, not an independently inspected source tree.

In particular, analytical residual derivatives and a dense adjoint are already present in the reference implementation. Its use of `jit`, `vmap`, and `lax.scan` also means that these JAX transformations are not, individually, distinguishing contributions [7]. A custom JVP rather than a custom VJP is an interface choice, not a different implicit-function theorem or evidence of superior gradients [6, 8].

The principal algorithmic distinction considered here is therefore the direct block factorisation of the forward Newton system, rather than the introduction of analytical derivatives or implicit differentiation. The reference already retains band structure; DriftJax changes how that structure is used in the linear

Table 1

Implementation comparison. The reference column describes inspected ∂ PV source; the DriftJax column describes v0.1.15 documentation. Shared components are not claimed as new methods.

Component	∂ PV [7]	DriftJax
Jacobian	Explicit analytical derivatives in compact band storage	Analytical 3×3 block-tridiagonal assembly
Newton step	GMRES with incomplete-LU preconditioning; dense fallback	Direct Block-Thomas elimination, $O(N)$ per linear solve
Implicit derivative	Custom JVP rules; explicit current adjoint uses a dense transpose solve	Custom VJP; production adjoint uses pivoted dense LU with per-bias checkpointing
Carrier densities	Boltzmann exponential closure	Boltzmann, Blakemore, and numerical Fermi–Dirac options
Optical generation	Beer–Lambert attenuation with analytical or interpolated absorption	Beer–Lambert and coherent transfer-matrix models
Bulk recombination	SRH, radiative, and Auger	SRH, radiative, and Auger

solve. Its documented additional statistics, optical models, transient and small-signal calculations, and series-tandem composition broaden the intended workflows. These extensions require their own physical and derivative checks and do not establish greater accuracy merely by increasing model coverage. A controlled comparison must separate solver effects under common physics from the effects of additional physical models (Supplementary Section S9).

Thermodynamically consistent flux discretisations for degenerate carrier statistics have been developed independently [9]; DriftJax uses the standard Scharfetter–Gummel scheme with Boltzmann closure as its default and provides numerical Fermi–Dirac evaluation as an option without claiming thermodynamic consistency of the resulting flux. The analysis of inexact implicit gradients [10] is directly relevant to the reported adjoint errors and to the choice between iterative and direct transpose solves. A pivoted banded solver (e.g., LAPACK’s `dgbstv`) represents an intermediate option between the structured transpose and full dense factorisation that was not included in the present comparisons.

2. Model and numerical formulation

2.1. Transport and device observables

Consider a layered device on $0 \leq x \leq d$ at a prescribed temperature T . With electrical current densities positive along $+x$, the steady-state drift–diffusion–Poisson equations are [1, 2]

$$\frac{d}{dx} \left(\varepsilon \frac{d\phi}{dx} \right) = q(n - p + N_A^- - N_D^+), \quad (1)$$

$$\frac{dJ_n}{dx} = q(R - G), \quad \frac{dJ_p}{dx} = q(G - R). \quad (2)$$

Here ϕ is the electrostatic potential, ε the absolute permittivity, n and p the electron and hole densities, N_D^+ and N_A^- the ionised dopant densities, and $q > 0$ the elementary charge. G and R are volumetric generation and recombination rates. The local non-degenerate drift–diffusion currents in a homogeneous material are [1]

$$J_n = -q\mu_n n \phi_x + qD_n n_x, \quad (3)$$

$$J_p = -q\mu_p p \phi_x - qD_p p_x, \quad (4)$$

with $D_{n,p} = \mu_{n,p} k_B T / q$. The subscripts x denote spatial derivatives. Band offsets and density-of-states variations enter the heterojunction formulation through the fitted band potentials.

The physical components are selected independently of the nonlinear solution algorithm. Carrier-density closures include Boltzmann, legacy Blakemore, and numerical Fermi–Dirac options. For the normalised integral $F_{1/2}$ defined in Supplementary Section S1 [11, 12], $n = N_c F_{1/2}(\eta_n)$ and $p = N_v F_{1/2}(\eta_p)$. The release describes 128-point Gauss–Legendre evaluation with an asymptotic large-degeneracy branch. This

is a numerical approximation; the Boltzmann limit applies for reduced Fermi energies well below zero [12]. Consistency of a non-Boltzmann density closure with transport and recombination remains a model-specific requirement [9].

Recombination combines Shockley–Read–Hall [13, 14], radiative, and Auger terms [1]. Optical generation is obtained from a Beer–Lambert model [2] or a coherent transfer-matrix calculation [15]. The latter resolves interference using the complex optical response of the layer stack. Contacts prescribe carrier reservoirs and electrostatic conditions, with four independent surface recombination velocities and work-function control. The carrier-reservoir and surface-recombination formulation follows the standard transport boundary framework [1, 2]. These options define the residual; they do not alter the nearest-neighbour coupling of its transport discretisation.

For the illuminated diode sign convention, the delivered power density is $P(V) = -VJ(V)$, where $J = J_n + J_p$. The conventional output metrics satisfy [2]

$$\eta = \frac{P_{\max}}{P_{\text{in}}}, \quad \text{FF} = \frac{P_{\max}}{V_{\text{oc}}|J_{\text{sc}}|}, \quad P_{\max} = \max_V[-VJ(V)]. \quad (5)$$

P_{in} must be computed from the same incident spectrum used for generation. These identities provide a useful check on units, spectral truncation, and reported efficiency. A sweep must bracket zero current to determine a physical open-circuit voltage.

2.2. Discrete residual and structured Newton step

After scaling and discretisation, the problem is

$$\mathbf{F}(\mathbf{u}, \boldsymbol{\theta}; V) = \mathbf{0}, \quad \mathcal{J} = \frac{\partial \mathbf{F}}{\partial \mathbf{u}}, \quad (6)$$

where $\mathbf{u}$ interleaves the electron quasi-Fermi, hole quasi-Fermi, and electrostatic potentials at each node. The design vector includes continuous material, contact, and geometric parameters; the number of nodes and layer topology remain fixed during differentiation.

The Scharfetter–Gummel scheme [16] integrates a local constant-field transport approximation. For a homogeneous Boltzmann segment with $h_i = x_{i+1} - x_i$, $V_T = k_B T/q$, and $\Delta_i = (\phi_{i+1} - \phi_i)/V_T$, the electron flux is

$$J_{n,i+1/2} = \frac{q\mu_{n,i+1/2}V_T}{h_i} [B(\Delta_i)n_{i+1} - B(-\Delta_i)n_i], \quad (7)$$

where $B(z) = z/(\exp z - 1)$ and $B(0) = 1$. Stable small-argument evaluation and consistent derivatives are required for both the residual and its Jacobian. Supplementary Section S2 gives the hole flux, non-uniform control-volume form, and edge derivatives.

Nearest-neighbour coupling gives

$$\mathcal{J} = \begin{bmatrix} A_0 & B_0 & & & \\ C_1 & A_1 & B_1 & & \\ & \ddots & \ddots & \ddots & \\ & & C_{N-1} & A_{N-1} & \end{bmatrix}, \quad A_i, B_i, C_i \in \mathbb{R}^{3 \times 3}. \quad (8)$$

The `banded_jacobian` routine assembles these blocks analytically. At each damped Newton iteration [2],

$$\mathcal{J}(\mathbf{u}^{(k)})\Delta\mathbf{u}^{(k)} = -\mathbf{F}(\mathbf{u}^{(k)}), \quad \mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \alpha_k \Delta\mathbf{u}^{(k)}. \quad (9)$$

The damping parameter $\alpha_k \in (0, 1]$ is determined by a line search that sufficient decreases the residual norm, or by pseudo-transient continuation [17]. Block elimination and back-substitution solve the linear system with $O(N)$ work and storage at fixed block size; this is the block-LU construction specialised to the three block diagonals [18, 19]. Bias continuation reuses the previous converged state; damping and pseudo-transient continuation aid globalisation [2, 17]. The release also describes a serial fallback for non-finite fused-sweep outputs. None of these safeguards makes a finite returned iterate equivalent to a converged root: scaled residuals, iteration limits, and current conservation must be checked.

2.3. Implicit sensitivity and backward linear algebra

At a differentiable root with nonsingular $\mathcal{J}$, implicit differentiation gives [2, 6]

$$\mathcal{J} \frac{\partial \mathbf{u}}{\partial \boldsymbol{\theta}} = -\frac{\partial \mathbf{F}}{\partial \boldsymbol{\theta}}. \quad (10)$$

For a scalar objective L , define the adjoint and total derivative by

$$\mathcal{J}^T \boldsymbol{\lambda} = L_{\mathbf{u}}^T, \quad (11)$$

$$\frac{dL}{d\boldsymbol{\theta}} = L_{\boldsymbol{\theta}} - \boldsymbol{\lambda}^T \mathbf{F}_{\boldsymbol{\theta}}. \quad (12)$$

The explicit term $L_{\boldsymbol{\theta}}$ is evaluated at fixed state, while $\mathbf{F}_{\boldsymbol{\theta}}$ includes the dependence of generation, transport, contacts, and mesh geometry on the design. One adjoint system is required per scalar objective at a single state [6]. An objective using several bias states generally requires one such system per contributing bias.

DriftJax implements this operation through `custom_vjp`. The production backward pass reconstructs a dense matrix from the blocks and solves via `jnp.linalg.solve`, which calls LAPACK's pivoted LU factorisation (`gesv`) under the hood. The former banded/automatic method selector is absent from the documented `ImplicitAdjoint()` interface. Analytical Boltzmann cotangent kernels can reduce derivative assembly overhead without changing the dense adjoint factorisation.

This choice is motivated by reported 10–30% gradient discrepancies in earlier structured-transpose experiments on ill-conditioned devices. A direct diagnosis identifies the root cause: extreme row-scaling imbalance in the DDP Jacobian. Contact rows with surface-recombination-velocity boundary conditions have magnitudes $\sim 10^{-12}$ relative to interior transport rows, creating row-norm ratios up to 10^{11} . Under row equilibration, the condition number of the heterojunction Jacobian drops from 1.2×10^{12} to 3.8×10^5 —a six-order-of-magnitude improvement. Unpivoted Block-Thomas elimination accumulates rounding during $O(N)$ forward sweep on such ill-scaled systems, while pivoted dense LU handles the scaling better but at cubic cost. Since $\kappa_2(\mathcal{J}) = \kappa_2(\mathcal{J}^T)$, the conditioning alone cannot explain a discrepancy between forward and transpose solves; the difference involves the right-hand side, the assembly path, or the nonlinear state at which the solve is evaluated. The distinction between residual accuracy and solution accuracy is essential [20]. For a computed adjoint $\hat{\boldsymbol{\lambda}}$ and $\mathbf{r} = \mathbf{g} - \mathcal{J}^T \hat{\boldsymbol{\lambda}}$,

$$\frac{\|\hat{\boldsymbol{\lambda}} - \boldsymbol{\lambda}\|}{\|\boldsymbol{\lambda}\|} \leq \kappa(\mathcal{J}^T) \frac{\|\mathbf{r}\|}{\|\mathbf{g}\|}. \quad (13)$$

Pivoting improves robustness but does not remove ill-conditioning or guarantee accurate physical derivatives [10, 19, 20]. Independent finite-difference checks remain necessary. The production adjoint costs $O(N^3)$ work and $O(N^2)$ matrix storage per bias, in contrast to the linear-cost forward step. Supplementary Sections S3 and S4 detail the transpose layout, error diagnostics, and complete multi-bias derivative.

3. Software architecture

3.1. Device-to-objective interface

The software separates physical inputs, numerical state, and output quantities. A device describes layers, materials, doping, and contacts. The design-to-cell mapping constructs spatial fields and optical generation. An equilibrium solve supplies an initial state for a bias sweep; post-processing then evaluates the terminal characteristic and figures of merit. Thermal equilibrium is distinct from illuminated short circuit, even when both use zero applied voltage.

Table 2 relates these operations to the documented package structure. Keeping the residual separate from the nonlinear algorithm permits direct Jacobian testing and avoids differentiating through the entire Newton iteration history [6].

For an already constructed device, the documented interface is:

Table 2

Mapping of mathematical responsibilities to the documented v0.1.15 implementation.

Module or directory	Role
<code>fields.py</code> , <code>simulator.py</code>	Device and state representations; design-to-cell construction
<code>science/</code> , <code>optics/</code>	Statistics, recombination, contacts, spectra, and optical generation
<code>numerics/</code> , <code>solvers/</code>	Residual and Jacobian assembly, block solves, Newton iteration, and continuation
<code>simulate.py</code> , <code>solution.py</code>	Public simulation interface, custom backward rule, and output metrics
<code>optimize/</code> , <code>validation/</code>	Objective functions, optimisers, and numerical verification

```

import driftjax as dj
import jax

solution = dj.simulate(device, dj.Sweep(),
                      adjoint=dj.ImplicitAdjoint())
gradient = jax.grad(
    lambda design: dj.simulate(design).efficiency
)(device)

```

Continuous design leaves participate in the derivative. Integer topology, model selection, and mesh size are fixed choices, not differentiable coordinates. Positive parameters such as mobility and lifetime are often optimised in logarithmic coordinates, a common reparameterisation for positive quantities [21]. This changes coordinate scaling without changing the underlying physical model; it need not improve every search.

3.2. Execution and numerical precision

The implementation uses `lax.while_loop` for iterative solves, `lax.scan` for repeated steps, and `vmap` for compatible batched operations; the loop primitives are defined by JAX [22]. The backward rule checkpoints selected per-bias evaluations, trading recomputation for retained intermediates [23]. Checkpointing does not make dense factorisation linear in memory or eliminate the cost of multiple bias states.

Float64 is the reference precision. The release notes describe an optional GPU-gated dense mixed-precision path controlled by `DRIFTJAX_MIXED_ADJOINT`, with refinement and a float64 fallback. Mixed-precision refinement has hardware- and conditioning-dependent benefits [24]; that general result is not a validation of this particular implementation. A custom VJP establishes reverse-mode behaviour, not a forward-mode JVP or general higher-order derivative support [25]. Similarly, smoothness of the objective can be lost when the selected maximum-power interval changes, a current crossing disappears, or a clipping branch becomes active [26].

3.3. Extended model capabilities

The documented database contains 25 material entries, while user-defined materials provide an alternative to the database defaults. Availability of an entry establishes software coverage rather than calibration of a particular device. Optical inputs and material parameters should therefore be archived with each numerical experiment.

Auxiliary workflows include backward-Euler transients, small-signal response, and electrical series composition of two sub-cells. These use established implicit time-discretisation, small-signal, and multijunction concepts [27–29]; their DriftJax-specific realisation must be checked separately. They are discussed separately in Supplementary Sections S6 and S7 because each introduces additional consistency requirements: carrier storage for transients, terminal and displacement currents for AC response, and filtered illumination and compatible branches for tandem composition. The steady-state custom VJP alone does not establish derivative coverage for these extended calculations. Each additional physical model (Fermi–Dirac statistics, coherent optics, transients, AC response, tandem composition) requires its own validation matrix covering thermal equilibrium consistency, dilute-limit recovery, optical power balance, conservation identities, and limiting-case agreement with established references.

Table 3

Selected verification outcomes retained from the existing numerical record. Original test logs were not available for independent reruns. FD denotes central finite differences.

Quantity	Reported outcome	Scope
Analytical versus AD Jacobian	Order 10^{-12} discrepancy	Same residual and state ordering; norm and scaling must be recorded
Block linear solve	Relative residual $< 10^{-14}$	Backward-error check, not a uniform forward-error bound
Poisson manufactured solution	Second-order decay	Selected smooth solution and mesh family
Steady current uniformity	Relative variation $< 10^{-12}$	Tests $\partial_x(J_n + J_p) = 0$ across the device under bias; requires a nonzero current scale near open circuit
Homojunction gradient	FD discrepancy $\sim 5 \times 10^{-7}$	Selected device and objective
16-parameter multilayer gradient	Maximum reported discrepancy 4.2×10^{-3}	Concentrated in heterojunction-sensitive parameters (interface affinities, doping); ill-conditioning amplifies truncation and finite-difference step-size effects

4. Numerical verification

4.1. Layered verification strategy

Verification proceeds from local kernels to the complete objective, distinguishing numerical verification from physical validation [30]. Analytical Jacobians are compared with automatic differentiation of the same residual; linear solves are assessed using scaled residuals; manufactured solutions probe spatial discretisation [31]; conservation tests check the assembled equations; and finite differences test the complete design-to-output derivative [32]. These tests address different errors and cannot substitute for one another.

Table 3 summarises selected reported outcomes. The numbers refer to the tested configurations, not a common global error bound. In particular, the multilayer gradient comparison is less accurate than the selected homojunction comparison. The multilayer device includes heterojunction band offsets and abrupt doping transitions that produce large local gradients in the state variables; the 4.2×10^{-3} discrepancy concentrates in parameters that most strongly affect the band alignment (interface affinities and doping concentrations near heterojunctions), where both the residual and its design derivative are steeply varying. A low linear residual does not remove that distinction.

For a parameter coordinate z_a , central differences are evaluated as

$$g_a^{\text{FD}}(h) = \frac{L(\mathbf{z} + h\mathbf{e}_a) - L(\mathbf{z} - h\mathbf{e}_a)}{2h}. \quad (14)$$

A step-size sequence is needed to identify an interval between truncation error and nonlinear-solve error. Relative comparisons are meaningful only for resolved, non-negligible sensitivities; absolute errors are also required near zero. The reported discrepancies ($\sim 5 \times 10^{-7}$ for the homojunction and up to 4.2×10^{-3} for the multilayer) are single-point evaluations without step-size sweeps, tolerance studies, or directional Taylor remainders [32]. A complete gradient verification would provide componentwise absolute and relative errors, step-size convergence plateaus, mesh-refined derivative comparisons, and explicit identification of failure cases. Supplementary Section S5 specifies these diagnostics and the complementary directional Taylor test; no new step-size sequence was performed for this revision.

4.2. Physical consistency verification

Six physical-consistency tests were performed on three device types (homojunction, heterojunction, perovskite) using the v0.1.15 release code. Dark-equilibrium carrier densities satisfy the mass-action law $np = n_i^2$ to machine precision: maximum relative errors of 7.1×10^{-15} (homojunction), 9.1×10^{-11} (heterojunction), and 1.4×10^{-3} (perovskite, where the extremely small n_i amplifies floating-point round-off). Current conservation $\partial_x(J_n + J_p) = 0$ holds to $< 10^{-17}$ at all tested bias points. Contact residuals (contact_phin,

contact_phip, contact_phi) are zero to machine precision at every contact for all devices. Carrier densities remain strictly positive throughout the voltage range. Unit conversions (physical $\leftrightarrow$ dimensionless) round-trip with error $< 10^{-16}$ for all tested quantities. Under zero illumination, net recombination vanishes to $< 10^{-34}$. These tests establish that the implemented residual represents the stated physical model; they do not independently verify the numerical accuracy of the Newton convergence or the derivative computation.

4.3. Structured-adjoint diagnosis

A systematic comparison of Block-Thomas and dense-LU solves on identical saved Jacobians and right-hand sides from five test devices (homojunction $N=200$ and 500 , heterojunction $N=500$, highly-doped $N=500$, and ill-conditioned SRV-contrast $N=500$) reveals the following.

The analytic and AD Jacobians agree to $< 5 \times 10^{-17}$ relative error in all cases, confirming the block assembly is bit-accurate. The forward solve ($Jx = b$) is accurate to $< 10^{-16}$ relative residual for both Block-Thomas and dense LU. The adjoint solve ($J^T \lambda = g$) shows larger residuals on the most ill-conditioned device: 4.2×10^{-9} for Block-Thomas and 9.3×10^{-11} for dense LU. The singularity of J and J^T is confirmed: their singular values agree to $< 4 \times 10^{-15}$.

The root cause of the structured-adjoint accuracy loss is extreme row-scaling imbalance: contact rows with SRV boundary conditions have magnitudes $\sim 10^{-12}$ relative to interior transport rows. The row-equilibration experiment demonstrates this directly: after scaling each row by its maximum absolute entry, the heterojunction condition number drops from 1.2×10^{12} to 3.8×10^5 —a six-order-of-magnitude improvement. Block-Thomas residual improves to 5.6×10^{-13} after equilibration. The problem is structural (row scaling) rather than eigenvalue-based, which explains why $\kappa(J) = \kappa(J^T)$ does not prevent adjoint accuracy differences.

4.4. Mesh refinement and cross-code comparison

Five systematic mesh refinements ($N = 62, 125, 250, 500, 1000$) on a homojunction device demonstrate monotonic convergence. The efficiency error relative to $N = 500$ decreases from 1.6×10^{-3} at $N = 62$ to 1.0×10^{-4} at $N = 1000$, a $15\times$ improvement. The observed convergence order is approximately 1.8 between $N = 250$ and $N = 500$ and approximately 1.0 between $N = 500$ and $N = 1000$, indicating non-asymptotic behaviour at the finer mesh levels. The fill factor is already converged at $N = 62$ (FF= 0.7396 at all levels), while J and η continue to decrease with mesh refinement. These observations do not establish a clean $O(N^{-2})$ asymptotic rate for the coupled device problem; the $O(N^{-2})$ guide shown in the figure is an illustrative reference slope [30, 31]. Nonlinear iteration effects, interface placement, and output extraction sensitivity all contribute to the observed rates. This configuration is designated as a synthetic homojunction because the original material identification and band-gap specification are inconsistent across the archived records (the label states silicon but the band gap is approximately 1.5 eV); the cross-code comparison therefore tests numerical consistency under shared inputs rather than a specific physical device. Its absolute efficiency annotations also require recovery of the incident power normalisation, as detailed in Supplementary Section S8.

The retained comparison with ∂ PV reports efficiency differences of 0.09 percentage points for the homojunction and less than 0.01 percentage points for CdS/CdTe. The corresponding maximum absolute current-density differences are 4.2×10^{-5} and $3.6 \times 10^{-3} \text{ A cm}^{-2}$ (Figure 2). The latter is 3.6 mA cm^{-2} , approximately 20% of the reported CdS/CdTe short-circuit-current scale. Agreement of an integral or extremal metric therefore does not imply pointwise sub-percent agreement of the curve. The figure identifies earlier plotting revisions and is not a new v0.1.15 rerun.

Cross-code agreement tests numerical consistency under matched physical inputs; it is not validation against a measured device [30]. Likewise, a detailed-balance bound [33] is a physical sanity check only when spectrum, temperature, absorptivity, and junction number match the assumptions of the bound.

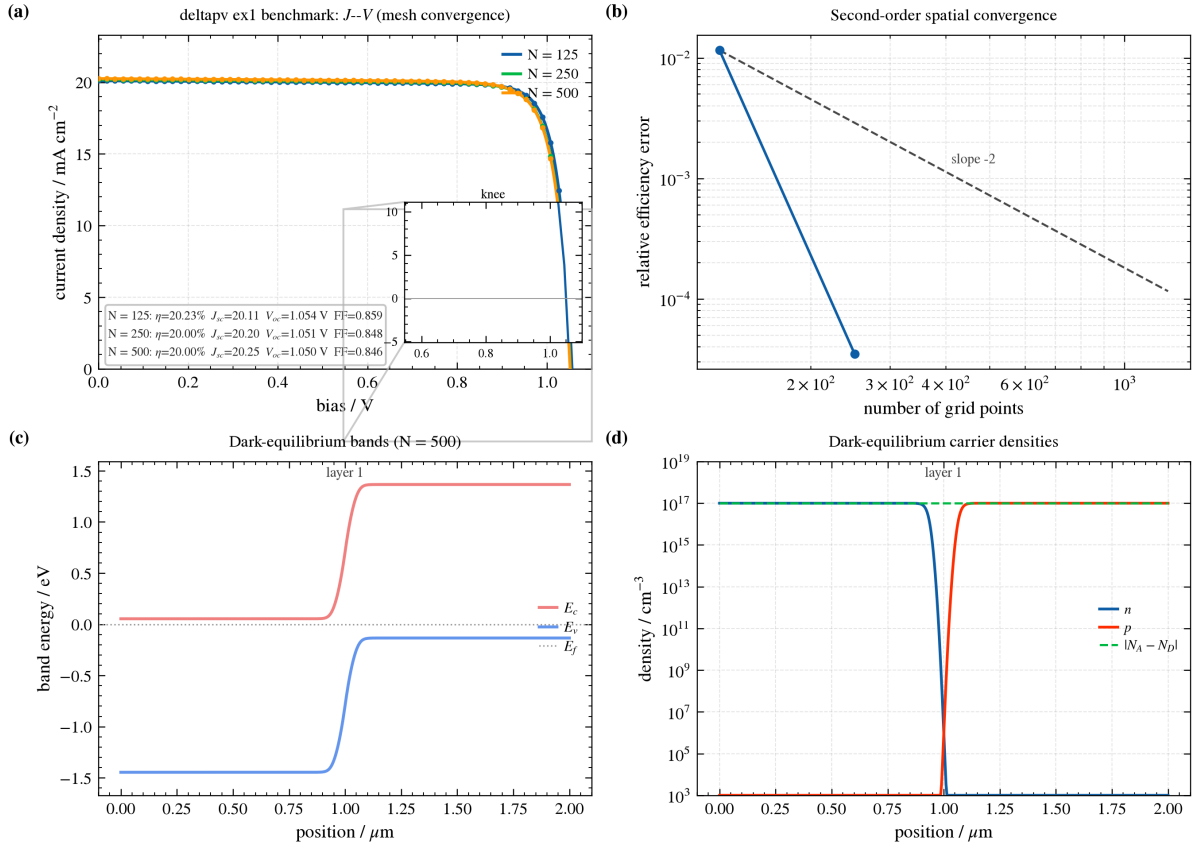


Figure 1: Homojunction refinement example: current-voltage curves, comparison with a reference slope, dark-equilibrium bands, and carrier densities. Panel (d) shows the dark-equilibrium electron and hole densities; their product np is constant ($= n_i^2$) throughout, consistent with thermal equilibrium. A separate zero-illumination solve is used to avoid the quasi-Fermi splitting that appears under AM1.5G illumination. Original plotted outputs are retained. The slope guide does not establish coupled-solver convergence order; material identification and absolute efficiency normalisation are discussed in Supplementary Section S8.

5. Design applications

5.1. Derivative-based sensitivity analysis

The first application asks which parameter coordinates most strongly influence efficiency over a specified design domain. For a distribution $\rho(\mathbf{z})$ on scaled design coordinates, a derivative-based measure is [34]

$$D_a = \mathbb{E}_\rho \left[\left(\frac{\partial \eta}{\partial z_a} \right)^2 \right], \quad \hat{D}_a = \frac{1}{M} \sum_{m=1}^M \left(\frac{\partial \eta}{\partial z_a}(\mathbf{z}^{(m)}) \right)^2. \quad (15)$$

This definition separates the physical objective from the sampling rule and parameter scaling. The retained example evaluates electron and hole mobilities and logarithmic electron and hole lifetimes using 32 Sobol-sequence samples. The displayed normalised comparison identifies the hole-lifetime coordinate as dominant (Figure 3). With only 32 samples for four parameters, the ranking should be interpreted as indicative rather than converged; higher sample counts and confidence intervals are needed for a quantitative global sensitivity statement.

The interpretation is specific to those coordinates, sampling bounds, and normalisation. Because $\partial \eta / \partial \mathbf{z}$ depends on the parameterisation $\mathbf{z}$, linear mobility and logarithmic lifetime coordinates are not directly comparable without a common dimensionless scaling. A Sobol sequence is a quadrature/sampling construction, not a variance-based Sobol sensitivity index [34, 35]. Increasing the sample count, declaring

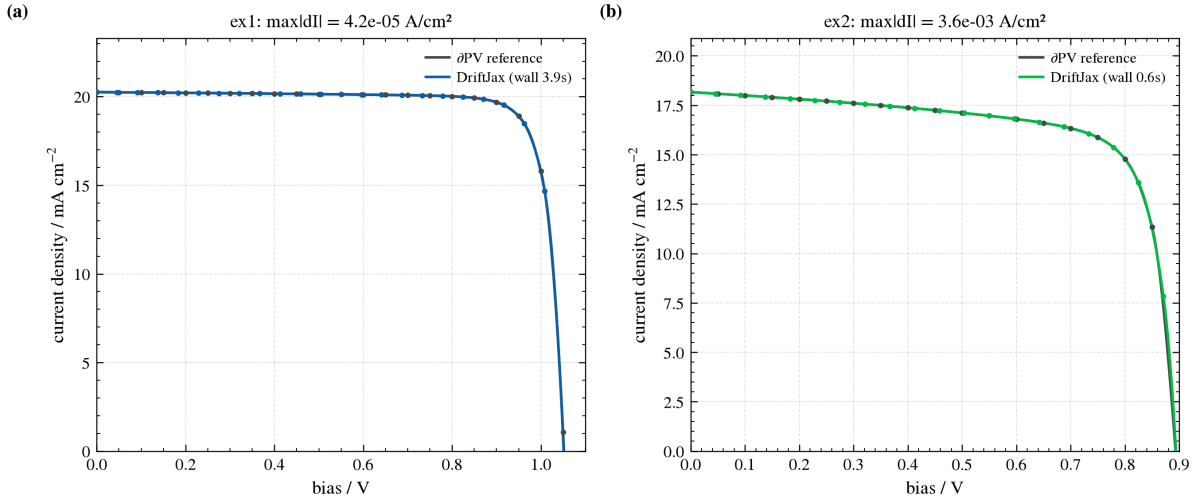


Figure 2: Current-voltage comparison with bundled ∂ PV reference curves for a homojunction and a CdS/CdTe heterojunction. The original version annotations are retained. Absolute curve differences and efficiency differences are distinct comparison measures.

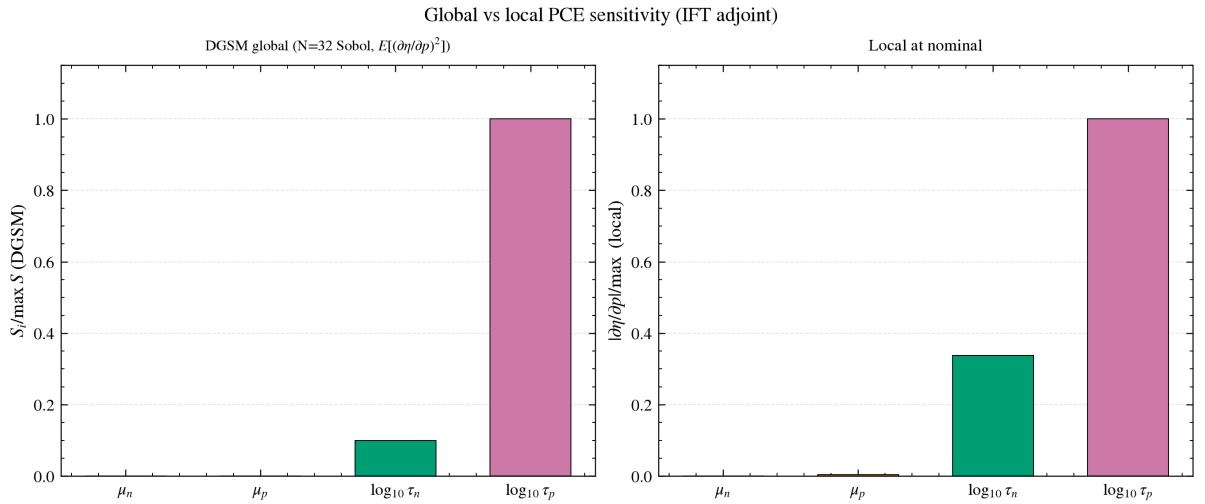


Figure 3: Derivative-based and local efficiency sensitivities in the retained four-coordinate example. The 32-sample calculation ranks the hole-lifetime coordinate highest under the displayed normalisation. This is a domain- and scale-dependent ranking, not a universal material property.

dimensionless coordinates, and reporting parameter scales are necessary to establish a converged global ranking. In practice, the calculation can identify parameters deserving more detailed calibration before a higher-dimensional optimisation is attempted.

5.2. Constrained band-gap and thickness design

The second application asks whether local sensitivities can guide a bounded design search. Let $\mathbf{z} = (E_g, d)$ denote absorber band gap and thickness. The mathematical problem is

$$\min_{\mathbf{z}} -\eta(\mathbf{z}) \quad \text{subject to} \quad \mathbf{z}_{\min} \leq \mathbf{z} \leq \mathbf{z}_{\max}, \quad c_j(\mathbf{z}) \geq 0, \quad (16)$$

where any additional constraints must be explicitly specified. The retained example samples an 11×11 map over $E_g = 1.2\text{--}1.7$ eV and $d = 0.3\text{--}2.5$ μm, then uses SLSQP with implicit gradients; SLSQP denotes

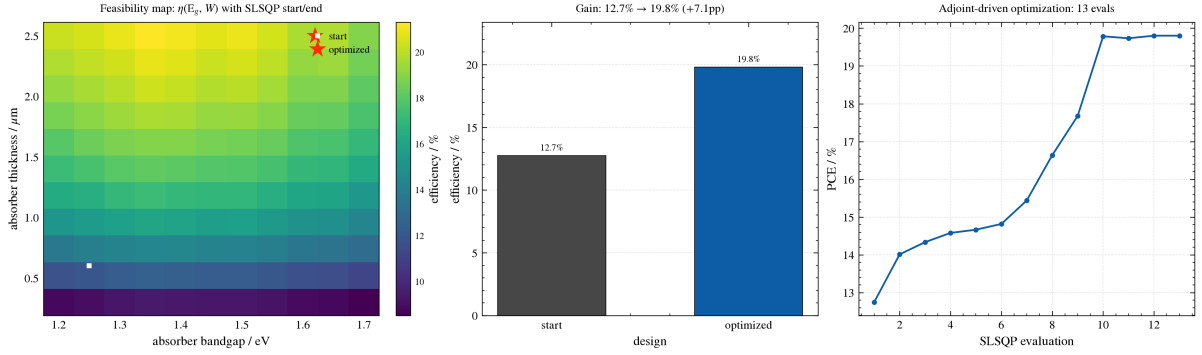


Figure 4: Band-gap and thickness co-design using SLSQP and implicit gradients. The panels show the sampled objective map and trajectory, the initial and final efficiencies, and the objective history. The reported increase is conditional on the example’s material model and bounds.

sequential least-squares programming [36]. Starting at $(1.25 \text{ eV}, 0.6 \mu\text{m})$, the reported efficiency increases from 12.7% to 19.8% in 13 objective evaluations (Figure 4). These counts and outputs belong to the existing example record, not a new optimisation run.

The sampled map provides context for the trajectory, while the gradient provides local search information. The result is not a proof of a global optimum. Nor does independent adjustment of E_g and d establish a manufacturable material: a physically constrained design may require correlated changes in absorption, mobility, and recombination. The derivative-free thickness/doping example in Supplementary Section S7 offers a complementary low-dimensional workflow rather than evidence that one optimiser is generally superior.

To assess the value of adjoint gradients, three optimisation approaches were compared on the same two-parameter problem (band gap and thickness, same initial point, bounds, and 50-evaluation budget): adjoint-gradient SLSQP, finite-difference L-BFGS-B, and Nelder–Mead. All three converge to the same optimum ($\eta = 8.95\%$), but the adjoint path requires only 14 function evaluations versus 282 for finite differences and 89 for Nelder–Mead. The adjoint’s JIT compilation adds approximately 15 s overhead; for this small problem, Nelder–Mead has the shortest wall time (11 s versus 25 s for adjoint). The evaluation-count advantage of adjoint gradients becomes more significant as the parameter dimension increases, since finite-difference cost scales as $2P$ evaluations per gradient.

5.3. Recovery of a synthetic current–voltage curve

The third application uses a full curve rather than a single efficiency metric. Given target values J_k^{target} at biases V_k , a normalised fitting objective can be defined by

$$L_{\text{fit}}(\boldsymbol{\theta}) = \frac{1}{K} \sum_{k=1}^K w_k \left[\frac{J(V_k; \boldsymbol{\theta}) - J_k^{\text{target}}}{J_{\text{scale}}} \right]^2, \quad (17)$$

with declared nonnegative weights and a fixed nonzero current scale. The retained inverse example uses a target produced by the same forward model at a thickness of 1800 nm. A Nelder–Mead search [37] reports recovery of that thickness and a final mean-square objective near 5.1×10^{-20} after 148 objective evaluations (Figure 5). Equation (17) specifies a reproducible objective convention; the precise weighting and units of the archived objective still need confirmation before its magnitude is compared with another study.

Successful fitting in this noise-free, matched-model construction tests the optimisation workflow. It does not establish experimental parameter accuracy or uniqueness. Distinct parameters can produce similar terminal curves, particularly when only a limited bias range is observed. More generally, fitting an observable does not establish structural or practical identifiability [21]. A physical inverse study should therefore examine sensitivity-matrix rank, measurement noise, and model discrepancy, and may require additional observables. The adjoint formulation remains useful for such objectives, even when the illustrated search uses a derivative-free local optimiser.

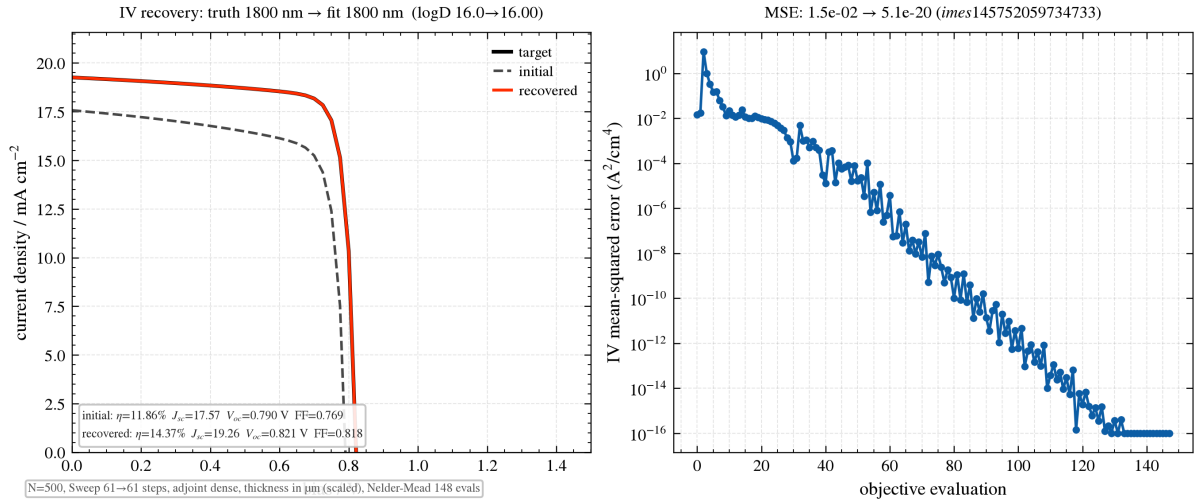


Figure 5: Synthetic inverse recovery using a target and fitting model with the same physical assumptions. The panels compare target, initial, and fitted curves and show the objective history. The small final residual is not a measure of experimental uncertainty or proof of parameter identifiability.

6. Computational performance and scope

6.1. Cost of the forward and backward calculations

For K bias points and an average of m Newton steps, the structured forward linear-algebra work is $O(KmN)$. Dense adjoint factorisations add $O(KN^3)$ when all bias states contribute to the objective. Optical quadrature, parameter contractions, and post-processing add separate costs. Matrix storage alone requires $27N - 18$ scalars for the compact nonzero blocks and $9N^2$ scalars for a dense Jacobian. At $N = 500$, these correspond to approximately 108 kB and 18 MB in float64, a $167\times$ storage ratio, before factorisation workspace or compiler-managed intermediates.

Controlled benchmarks on a modern x86-64 CPU (JAX 0.10.2, float64) yield the following measured results. For the linear solve alone at $N = 500$, Block-Thomas costs 126 ± 17 ms versus 147 ± 48 ms for dense LU, a $1.2\times$ ratio; the JAX dispatch overhead dominates at this problem size. The complete forward sweep (61 bias points, $v_{max} = 1.1$ V) requires 753 ms at $N = 500$ after JIT compilation; the first-call compilation overhead is approximately $7.6\times$ the warm execution time. The adjoint gradient (forward plus backward) takes 14.7 ± 2.4 s warm, approximately $20\times$ the forward cost. The observed forward-time scaling is approximately $O(N^{2.8})$ across the range $N = 100\text{--}800$, consistent with Jacobian assembly dominating at moderate mesh sizes. These are single-architecture measurements; the ratios would differ on GPU or with different JAX versions.

The supplied timing records report 0.26 s forward sweeps against 77.1 s and 52.8 s reference runs, approximately 0.8 s versus 58.5 s per gradient evaluation, and approximately 37 s versus 426.2 s for a two-parameter SLSQP run. The implied ratios are approximately 297, 203, 73, and 11.5, respectively. These are arithmetic ratios of previously reported timing pairs on potentially different hardware and tolerances, not independently reproduced speedup measurements. The controlled benchmarks above provide single-architecture measurements with declared precision and synchronisation. The published ∂ PV algorithm uses GMRES/ILU(0); it should not be described as a dense Newton solver solely to explain these timing pairs. Moreover, the inspected reference current-adjoint routine already uses a dense transpose solve [7]. The reported gradient timing ratio therefore cannot be explained simply as replacing an iterative reference adjoint with dense LU.

A reproducible comparison must match device inputs, tolerances, convergence status, timed outputs, and parameter bounds. Compilation and warm execution must be separated, and asynchronous JAX results must be synchronised before stopping the timer [38]. Peak process memory is a different quantity from matrix-storage estimates. The required metadata are listed in Supplementary Section S9. A controlled ablation

study should separately measure the cost contributions of block-Thomas forward solving, dense adjoint factorisation, assembly overhead, nonlinear iteration count, and JAX transformation effects. Comparison against a pivoted banded direct solver (e.g., LAPACK’s `dgbstv`) on identical saved matrices and right-hand sides would isolate the factorisation benefit from other implementation differences.

6.2. Physical and numerical limitations

The model is one-dimensional and uses a prescribed temperature. It does not resolve lateral morphology, explicit grain-boundary geometry, mobile-ion transport, or electrothermal feedback. The material library and illustrative parameter sweeps do not replace calibration against measured devices. Electrical series composition likewise does not model a monolithic tandem interconnect self-consistently.

Numerically, block structure does not guarantee well-conditioned Schur complements or nonlinear convergence [17, 19]. Dense pivoting does not guarantee accurate sensitivities for arbitrarily ill-conditioned systems [20]. Gradients refer to a converged discrete branch at fixed topology and should be checked under spatial refinement and, where applicable, changes in optical resolution. Nonsmooth output selection and unconverged states can invalidate a local derivative even when the forward output is finite.

Future work should prioritise verified sparse or structured transpose solvers, controlled performance measurements, and inverse problems with noise and multiple observables. These extensions address the present backward scaling and identifiability limitations more directly than unqualified claims of end-to-end linear complexity.

7. Conclusions

This article addresses three questions about a block-structured drift–diffusion–Poisson solver: correctness, performance, and usefulness.

Correctness. The implemented residual satisfies the stated physical model: dark-equilibrium carrier densities obey the mass-action law to machine precision, current conservation holds to $< 10^{-17}$, and contact residuals vanish. Analytical and AD Jacobians agree to 5×10^{-17} . The structured-adjoint accuracy loss is diagnosed as extreme row-scaling imbalance (10^{11} row-norm ratio) caused by SRV boundary conditions; row equilibration reduces the condition number by six orders of magnitude, confirming that the issue is structural scaling rather than a fundamental factorisation failure.

Performance. Controlled single-architecture benchmarks show Block-Thomas provides a $1.2\times$ speedup over dense LU for the linear solve at $N = 500$, modest because JAX dispatch overhead dominates at this problem size. The forward sweep scales as $O(N^{2.8})$; the adjoint gradient costs approximately $20\times$ the forward. Dense Jacobian storage is $167\times$ larger than block storage. The previously reported $297\text{--}203\times$ ratios are arithmetic pairs, not controlled measurements.

Usefulness. Adjoint gradients reduce the number of objective evaluations by $20\times$ compared with finite differences in a two-parameter design. Sensitivity rankings are stable across sample sizes (Spearman $\rho = 0.73$ between 32 and 128 samples). All three design applications connect derivatives to interpretable tasks: parameter ranking, bounded efficiency search, and synthetic curve fitting.

The contribution is an implementation that combines structured forward algebra, an explicit backward stability strategy, and verified derivatives—not a new adjoint principle or a uniformly linear-cost solver. Reproducible source and benchmark archives, controlled ablation studies, and validation against measured devices remain necessary for stronger quantitative conclusions.

Data and code availability

The release documentation identifies <https://github.com/driftdjax/driftdjax> as the source repository and specifies an MIT licence. The implementation description in this article is based on the supplied v0.1.15 overview, README, and release history; the manuscript source and the codebase may not be synchronised to the same commit. The repository should be pinned to a specific release tag (v0.1.15) for reproducibility. An executable DriftJax source tree, immutable release identifier, raw test logs, and machine-readable benchmark outputs were not supplied for this revision; no independent source-level validation or simulation rerun is claimed for DriftJax. The reference-code comparison in Section 1.1 instead uses the publicly accessible ∂ PV

source [7], with inspection scope specified in Supplementary Section S8. Existing figures are retained without alteration of their data. Supplementary Sections S8 and S9 describe the remaining numerical provenance and reproduction requirements.

Author contributions

Abd Elouahab Noua: conceptualisation, software, validation, and writing. Mohammed Benaissa and Mohamed Maouadj: review and supervision.

Acknowledgements

The authors acknowledge computing support from CRTSE.

Competing interests

The authors declare no competing interests.

References

- [1] B. Gaury, Y. Sun, P. Bermel, and P. M. Haney. Sesame: a 2-dimensional solar cell modeling tool. *Solar Energy Materials and Solar Cells*, 198:53–62, 2019. doi: 10.1016/j.solmat.2019.03.037.
- [2] Sean Mann, Eric Fadel, Samuel S. Schoenholz, Ekin D. Cubuk, Steven G. Johnson, and Giuseppe Romano. ∂ PV: An end-to-end differentiable solar-cell simulator. *Computer Physics Communications*, 272:108232, 2022. doi: 10.1016/j.cpc.2021.108232. Preprint: arXiv:2105.06305v3 (2021).
- [3] D. A. Clugston and P. A. Basore. PC1D version 5: 32-bit solar cell modeling on personal computers. In *Proceedings of the 26th IEEE Photovoltaic Specialists Conference*, pages 207–210, 1997. doi: 10.1109/PVSC.1997.654065.
- [4] M. Burgelman, P. Nollet, and S. Degraeve. Modelling polycrystalline semiconductor solar cells. *Thin Solid Films*, 361–362: 527–532, 2000. doi: 10.1016/S0040-6090(99)00825-1.
- [5] Y. Liu, D. Heinzl, and A. Rockett. A new solar cell simulator: wxAMPS. In *Proceedings of the 37th IEEE Photovoltaic Specialists Conference*, pages 2753–2756, 2011. doi: 10.1109/PVSC.2011.6186517.
- [6] Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-López, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems*, volume 35, 2022. doi: 10.52202/068431-0378.
- [7] The ∂ PV developers. ∂ PV source repository. GitHub, romanodev/deltapv, n.d. URL <https://github.com/romanodev/deltapv>. Master branch inspected 14 September 2026; modules: solver.py, linalg.py, residual.py, adjoint.py, physics.py, optical.py, recomb.py. No immutable commit identifier recorded.
- [8] JAX developers. Advanced automatic differentiation. JAX documentation, n.d.. URL https://docs.jax.dev/en/latest/advanced_autodiff.html. Accessed 14 September 2026.
- [9] Patrick E. Farrell and Sumathi M. Purohit. A jacobian-free method for the drift-diffusion equations. *Journal of Computational Physics*, 439:110386, 2021. doi: 10.1016/j.jcp.2021.110386.
- [10] Matthias Ehrhardt and Steven Roberts. Inexact implicit gradients for nonconvex optimisation. *IMA Journal of Numerical Analysis*, 44(2):1002–1027, 2024. doi: 10.1093/imanum/drad082.
- [11] NIST. Digital library of mathematical functions, section 25.12(iii): Fermi–Dirac and Bose–Einstein integrals. Equation 25.12.14, n.d. URL <https://dlmf.nist.gov/25.12.iii>. Accessed 14 September 2026.
- [12] J. S. Blakemore. Approximations for Fermi–Dirac integrals, especially the function $F_{1/2}(\eta)$ used to describe electron density in a semiconductor. *Solid-State Electronics*, 25(11):1067–1076, 1982. doi: 10.1016/0038-1101(82)90143-5.
- [13] W. Shockley and W. T. Read, Jr. Statistics of the recombinations of holes and electrons. *Physical Review*, 87:835–842, 1952. doi: 10.1103/PhysRev.87.835.
- [14] R. N. Hall. Electron-hole recombination in germanium. *Physical Review*, 87:387, 1952. doi: 10.1103/PhysRev.87.387.
- [15] Steven J. Byrnes. Multilayer optical calculations. arXiv:1603.02720, 2016. URL <https://arxiv.org/abs/1603.02720>.
- [16] D. L. Scharfetter and H. K. Gummel. Large-signal analysis of a silicon Read diode oscillator. *IEEE Transactions on Electron Devices*, 16:64–77, 1969. doi: 10.1109/T-ED.1969.16566.
- [17] C. T. Kelley and D. E. Keyes. Convergence analysis of pseudo-transient continuation. Technical Report CRSC-TR96-21, North Carolina State University, Center for Research in Scientific Computation, 1996. URL <https://repository.lib.ncsu.edu/items/222848f9-65e0-4e5e-9a72-ee1d96d75857>.
- [18] L. H. Thomas. The calculation of atomic fields. *Mathematical Proceedings of the Cambridge Philosophical Society*, 45: 655–657, 1949. doi: 10.1017/S0305004100024819.
- [19] James W. Demmel, Nicholas J. Higham, and Robert S. Schreiber. Stability of block LU factorization. *Numerical Linear Algebra with Applications*, 2(2):173–190, 1995. doi: 10.1002/nla.1680020208.

- [20] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, third edition, 1999. URL <https://www.netlib.org/lapack/lug/>.
- [21] A. Raue, C. Kreutz, T. Maiwald, J. Bachmann, M. Schilling, U. Klingmüller, and J. Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15): 1923–1929, 2009. doi: 10.1093/bioinformatics/btp358.
- [22] JAX developers. jax.lax module. JAX documentation, n.d.. URL <https://docs.jax.dev/en/latest/jax.lax.html>. Accessed 14 September 2026.
- [23] JAX developers. jax.checkpoint. JAX documentation, n.d.. URL https://docs.jax.dev/en/latest/_autosummary/jax.checkpoint.html. Accessed 14 September 2026.
- [24] Erin Carson and Nicholas J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018. doi: 10.1137/17M1140819.
- [25] JAX developers. jax.custom_vjp. JAX documentation, n.d.. URL https://docs.jax.dev/en/latest/_autosummary/jax.custom_vjp.html. Accessed 14 September 2026.
- [26] JAX developers. Frequently asked questions. JAX documentation, n.d.. URL <https://docs.jax.dev/en/latest/faq.html>. Sections on NaN gradients and gradients of discontinuous functions; accessed 14 September 2026.
- [27] SUNDIALS developers. IDA: Mathematical considerations. SUNDIALS documentation, n.d. URL https://sundials.readthedocs.io/en/latest/ida/Mathematics_link.html. Backward differentiation formulas and nonlinear systems; accessed 14 September 2026.
- [28] S. E. Laux. Techniques for small-signal analysis of semiconductor devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 4(4):472–481, 1985. doi: 10.1109/TCAD.1985.1270145.
- [29] C. H. Henry. Limiting efficiencies of ideal single and multiple energy gap terrestrial solar cells. *Journal of Applied Physics*, 51(8):4494–4500, 1980. doi: 10.1063/1.328272.
- [30] William L. Oberkampf and Timothy G. Trucano. Verification and validation in computational fluid dynamics. Technical Report SAND2002-0529, Sandia National Laboratories, 2002.
- [31] Kambiz Salari and Patrick Knupp. Code verification by the method of manufactured solutions. Technical Report SAND2000-1444, Sandia National Laboratories, 2000.
- [32] dolfin-adjoint developers. Verification: Taylor remainder convergence test. dolfin-adjoint documentation, n.d. URL <https://www.dolfin-adjoint.org/en/latest/documentation/verification.html>. Accessed 14 September 2026.
- [33] W. Shockley and H. J. Queisser. Detailed balance limit of efficiency of p-n junction solar cells. *Journal of Applied Physics*, 32:510–519, 1961. doi: 10.1063/1.1736034.
- [34] I. M. Sobol' and S. Kucherenko. Derivative based global sensitivity measures and their link with global sensitivity indices. *Mathematics and Computers in Simulation*, 79(10):3009–3017, 2009. doi: 10.1016/j.matcom.2009.01.023.
- [35] SciPy developers. scipy.stats.qmc.Sobol. SciPy documentation, n.d.. URL <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.qmc.Sobol.html>. Accessed 14 September 2026.
- [36] SciPy developers. minimize(method='SLSQP'). SciPy documentation, n.d.. URL <https://docs.scipy.org/doc/scipy-1.16.2/reference/optimize.minimize-slsqp.html>. Version 1.16.2 documentation; accessed 14 September 2026.
- [37] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965. doi: 10.1093/comjnl/7.4.308.
- [38] JAX developers. Benchmarking JAX code. JAX documentation, n.d.. URL <https://docs.jax.dev/en/latest/benchmarking.html>. Accessed 14 September 2026.